128. Longest Consecutive Sequence

Medium · Array · Hash Table · Union Find

Approach — Hash Set + Sequence Start Detection

Sorting the array would give an O(n log n) solution, but using a **hash set** for O(1) lookups allows the same problem to be solved in O(n). All numbers are first inserted into an `unordered_set`, removing duplicates and enabling instant existence checks.

The crucial trick is to only start counting a sequence from its **true beginning** — a number n is the start of a sequence only if `n-1` is *not* in the set. For every such starting number, the code walks forward (`n, n+1, n+2, ...`) counting how long the consecutive run extends, and tracks the maximum length found.

Key Insight: By skipping any number that has a predecessor in the set, each consecutive sequence is explored exactly once — starting from its smallest element. This guarantees the total work across all sequences sums to O(n), even though there's a while loop inside the for loop.

Step-by-step Breakdown

Step 1 — Build a hash set of all numbers

Insert every element of `nums` into `unordered_set s`. This removes duplicates and provides O(1) average-time membership checks via `s.count(...)`.

Step 2 — Identify sequence starting points

Iterate over every number `n` in the set. If `s.count(n-1)` is true, n is not the start of a sequence — some earlier element will eventually reach and count it, so `continue` to skip it.

Step 3 — Walk forward to measure the sequence length

For a valid starting number n, initialize `curr = n` and `len = 0`. While `s.count(curr)` is true, increment both `curr` and `len` — effectively counting how many consecutive integers starting from n exist in the set.

Step 4 — Track the maximum length

After the inner while loop finishes, update `mx = max(mx, len)` to record the longest consecutive run found so far. After processing all starting points, return `mx` as the final

answer.

Solution Code

```cpp
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> s(nums.begin(), nums.end());

        int mx = 0;
        for(auto &n: s)
        {
            if(s.count(n-1)) continue;
            int curr = n, len = 0;
            while(s.count(curr)) ++curr, ++len;
            mx = max(mx, len);
        }
        return mx;
    }
};
```

Complexity Analysis

TIME COMPLEXITY	SPACE COMPLEXITY
O(n)	**O(n)**
Building the set is O(n). Although there's a nested while loop, each element is only ever visited as part of the while loop once — because the outer loop skips non-starting numbers — so total work across all iterations is O(n).	The hash set stores up to n distinct numbers from the input array, giving O(n) auxiliary space.